

The Most Efficient Protocol for PAKE: What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Preliminaries

3 Security Analysis of the “Hash Everything” Version

3.1 The Simulator

3.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P -to- P' message. When P 's session begins, $\mathcal{S}$ must simulate its message (s, T) to P' . This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\text{pw}^*, T)$ (let the result be t^*), $\mathcal{S}$ needs to generate a KEM key pair (pk^*, sk^*) for later use (see the “Extracting password guess from malicious P' ” paragraph below) and make sure that (pk^*, sk^*) is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*,$$

and programs $H_1(\text{pw}^*, r^*) := R^*$. This also allows $\mathcal{S}$ to store the association between pw^* and (pk^*, sk^*) . (Note that (pk^*, sk^*) needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R^* never changes and $\mathcal{A}$ can easily find collisions in H_1 .)

Extracting password guess from malicious P . When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P' , if it is different from the honest message (s, T) , the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding

TestPwd command to $\mathcal{F}_{\text{PAKE}}$. There are two types of attack, which need to be addressed separately:

1. “Normal” online attack: $\mathcal{A}$ executes P ’s algorithm on a password guess pw^* . That is, $\mathcal{A}$ samples public key pk^* and randomness r^* , computes

$$\begin{aligned} R^* &:= H_1(\text{pw}^*, r^*), & T^* &:= pk^* \cdot R^*, \\ t^* &:= H_2(\text{pw}^*, T^*), & s^* &:= t^* \oplus r^*, \end{aligned}$$

and sends (s^*, T^*) to P' .

It takes a while to realize how to extract pw^* from (s^*, T^*) . First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the “actual” password guess. This can be done via the following. For each such (pw^*, t^*) pair, $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\text{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\text{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the “actual” password guess pw^* . (This special $H_1(\text{pw}^*, r^*)$ query is called the “anchor query” in [MRR20, JRX25].)

2. *Related key attack*: This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P . $\mathcal{A}$ skips sampling pk^* or r^* ; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$\begin{aligned} t^* &:= H_2(\text{pw}^*, T), & (t^*)' &:= H_2(\text{pw}^*, T^*), \\ \delta &:= t^* \oplus (t^*)', & s^* &:= s \oplus \delta, \end{aligned}$$

and sends (s^*, T^*) to P' .

Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*) , such that

$$s \oplus s^* = H_2(\text{pw}, T) \oplus H_2(\text{pw}, T^*),$$

which allows $\mathcal{A}$ to pick T^* and “reverse engineer” the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\text{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.

To extract pw^* from (s^*, T^*) , $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which ones satisfy

$$H_2(\text{pw}^*, T) \oplus H_2(\text{pw}^*, T^*) = s \oplus s^*.$$

With overwhelming probability, at most one pw^* will satisfy this equation.

Extracting password guess from malicious P' . When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P , if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ and $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding TestPwd command to $\mathcal{F}_{\text{PAKE}}$.

$\mathcal{S}$ should find the H_{tag} query whose result is τ^* , which includes (pw^*, pk^*, K^*) . However, pw^* constitutes a password guess only if it is consistent with pk^* and K^* , i.e., $K^* = \text{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^* . $\mathcal{S}$ can check this condition because it has stored the correspondence between pw^* and (pk^*, sk^*) (see the “Simulating honest P -to- P' message” paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if *any* of pw^*, pk^*, c^* is not included in H_{tag} . In later sections we will explain how to simulate other versions of the protocol.)

3.1.2 Formal Description of the Simulator

3.2 Hybrid Argument

We now prove that for any PPT environment $\mathcal{Z}$, the simulator $\mathcal{S}$ in Section 3.1.2 generates an ideal-world view indistinguishable from $\mathcal{Z}$ ’s real-world view. The sequence of games start from the real world (Game 0) and ends at the ideal world.

Game 0: This is the real world. Recall that the passwords of P and P' are pw and pw' , respectively; the flow of events is:

- P sends (s, T) to P' ;
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who sends (s^*, T^*) (which may or may not be equal to (s, T)) to P' ;
- P' outputs session key SK' and sends (c, τ') to P ;
- It is again intercepted by $\mathcal{A}$, who sends (c^*, τ^*) to P ;
- P computes τ . If $\tau^* = \tau$ then P outputs SK , otherwise P outputs $\perp$.

(Note that there are three τ values: τ' computed and sent by P' , τ^* sent by $\mathcal{A}$, and τ computed by P used in P ’s check.)

Game 1: In the case where $\text{pw} = \text{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is passive), set $SK := SK'$.
- If $\tau^* \neq \tau'$, set $SK := \perp$.

By correctness of the protocol (see Section 2), in Game 0 $\tau = \tau'$ except with probability $\text{Adv}^{\text{correctness}}$. This means that except with probability $\text{Adv}^{\text{correctness}}$, P ’s check $\tau^* \stackrel{?}{=} \tau$ in Game 0 is equivalent to P ’s check $\tau^* \stackrel{?}{=} \tau'$ in Game 1. If the check passes then both games set $SK := H_{\text{key}}(\text{pw}, pk, c, K)$, otherwise both games set $SK := \perp$. We have

$$\text{Dist}_{\mathcal{Z}}^{0,1} = \text{Adv}^{\text{correctness}}.$$

Game 2: In the case where $\tau^* \neq \tau'$, set $SK := \perp$ if either of the following happens:

- $\mathcal{A}$ never makes an $H_{\text{tag}}(\text{pw}, pk, c^*, \star)$ query resulting in τ^* , or
- $\mathcal{A}$ makes two different $H_{\text{tag}}(\text{pw}, pk, c^*, \star)$ queries, both resulting in τ^* .

In the first event, Game 1 and Game 2 differ if $\mathcal{A}$ never makes an $H_{\text{tag}}(\text{pw}, pk, c^*, \star)$ query resulting in τ^* , yet P 's check $\tau^* \stackrel{?}{=} H_{\text{tag}}(\text{pw}, pk, c^*, K)$ passes. Clearly,

- $\mathcal{A}$ does not query $H_{\text{tag}}(\text{pw}, pk, c^*, K)$ (otherwise the result is τ^*), and
- P' does not query $H_{\text{tag}}(\text{pw}, pk, c^*, K)$ (otherwise $\tau' = H_{\text{tag}}(\text{pw}, pk, c^*, K) = \tau^*$).

So $H_{\text{tag}}(\text{pw}, pk, c^*, K)$ is independent of $\mathcal{A}$'s view, and the probability that $\mathcal{A}$ sends $\tau^* = H_{\text{tag}}(\text{pw}, pk, c^*, K)$ is $1/2^n$. The second event is collision in H_{tag} , whose probability is upper-bounded by $q_{\text{tag}}(q_{\text{tag}} - 1)/2^{n+1}$. Overall,

$$\mathbf{Dist}_{\mathcal{Z}}^{1,2} \leq \frac{q_{\text{tag}}(q_{\text{tag}} - 1) + 2}{2^{n+1}}.$$

Game 3: In the case where $\text{pw} = \text{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0, 1\}^n$. (c is still computed as in Game 2.)

Let Query_1 be the event that $\mathcal{A}$ queries *either* $H_{\text{key}}(\text{pw}, pk, c, K')$ or $H_{\text{tag}}(\text{pw}, pk, c, K')$. Clearly Game 2 and Game 3 are identical unless Query_1 happens. We upper-bound $\Pr[\text{Query}_1]$ via a reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property of KEM:

Reduction $\mathcal{R}_{\text{OW-1PCA1}}$: // Boxed text indicates the single PC oracle query; same for reductions below

Sample $i \leftarrow [q_{\text{key}} + q_{\text{tag}}]$ as a guess that $\mathcal{A}$'s i -th H_{key} or H_{tag} query triggers Query_1 .

On input (pk, c) ,

1. Invoke $\mathcal{Z}$. On $(\text{NewSession}, \text{sid}, P, P', \text{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P 's algorithm to compute (s, T) . (Note that computing (s, T) does not require knowing sk .) Send (s, T) to $\mathcal{A}$.
2. On $(\text{NewSession}, \text{sid}, P', P, \text{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\text{pw} = \text{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0, 1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW-PCA1}}$'s inputs.)
3. On (c^*, τ^*) from $\mathcal{A}$,
 - (a) If $c^* = c$, do what's described in Game 1.
 - (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\text{tag}}(\text{pw}, pk, c^*, K^*)$ query whose result is τ^* . If there is none or more than one, set $SK := \perp$. (This matches Game 2.)

(c) $\boxed{\text{Query PC}(sk, c^*, K^*)}$.

If the result is 1, output $SK := H_{\text{tag}}(\text{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.

If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. On $\mathcal{A}$'s i -th H_{key} or H_{tag} query, if it is in the form of $H_{\text{key}}(\text{pw}, pk, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_1 happens, the only difference between Game 2/3 and $\mathcal{R}_{\text{OW-1PCA1}}$'s simulation (until Query_1 happens) is that $\mathcal{R}_{\text{OW-1PCA1}}$ uses its input pk on the P side, and then uses its input $c \leftarrow \text{Encap}(pk)$ on the P' side. Since we assume $\text{pw} = \text{pw}' \wedge (s^*, T^*) = (s, T)$, in Game 2/3 P' 's algorithm will recover $pk' = pk$, so there c is also generated as $\text{Encap}(pk)$ and thus $\mathcal{R}_{\text{OW-1PCA1}}$ simulates Game 2/3 perfectly. Furthermore, if $\mathcal{R}_{\text{OW-1PCA1}}$'s guess is correct then it wins the OW-1PCA game. We have

$$\text{Dist}_{\mathcal{Z}}^{2,3} \leq \Pr[\text{Query}_1] \leq (q_{\text{key}} + q_{\text{tag}}) \text{Adv}_{\mathcal{R}_{\text{OW-1PCA1}}}^{\text{OW-1PCA}}.$$

Game 4: Again in the case where $\text{pw} = \text{pw}' \wedge (s^*, T^*) = (s, T)$, generate P' 's KEM public key $pk' \leftarrow \text{Gen}$ and compute $(c, \star) \leftarrow \text{Encap}(pk')$. (Note that K' is not used anywhere since Game 3 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.)

The difference between Game 3 and Game 4 is that in Game 3 both P and P' use the same pk , whereas in Game 4 P uses pk and P' uses an independent pk' . We construct a reduction $\mathcal{R}_{\text{ANO-1PCA1}}$ to the ANO-1PCA property of KEM. This reduction, in fact, is very similar to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3, and we highlight their differences in blue:

Reduction $\mathcal{R}_{\text{ANO-1PCA1}}$:

On input (pk, c) ,

1. Invoke $\mathcal{Z}$. On $(\text{NewSession}, sid, P, P', \text{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P 's algorithm to compute (s, T) , **except that $\mathcal{R}_{\text{ANO-1PCA1}}$ uses its input pk rather than $(pk, \star) \leftarrow \text{Gen}$** . (Note that computing (s, T) does not require knowing sk .) Send (s, T) to $\mathcal{A}$.
2. On $(\text{NewSession}, sid, P', P, \text{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\text{pw} = \text{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0, 1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO-1PCA1}}$'s inputs.)
3. On (c^*, τ^*) from $\mathcal{A}$,
 - (a) If $c^* = c$, do what's described in Game 1.
 - (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\text{tag}}(\text{pw}, pk, c^*, K^*)$ query whose result is τ^* . If there is none or more than one, set $SK := \perp$. (This matches Game 2.)

- (c) Query $\text{PC}(sk, c^*, K^*)$. // This is a valid query since we do this step only if $c^* \neq c$
 If the result is 1, output $SK := H_{\text{tag}}(\text{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
 If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. When $\mathcal{Z}$ outputs bit b^* , $\mathcal{R}_{\text{ANO-1PCA1}}$ also outputs b^* .

$\mathcal{R}_{\text{ANO-1PCA1}}$ uses its input pk on the P side and c on the P' side. If $b = 0$ then $c \leftarrow \text{Encap}(pk)$, i.e., both P and P' use the same pk , so $\mathcal{R}_{\text{ANO-1PCA1}}$ simulates Game 3; if $b = 1$ then $c \leftarrow \text{Encap}(pk')$, i.e., P' uses an independently generated pk' , so $\mathcal{R}_{\text{ANO-1PCA1}}$ simulates Game 4. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{3,4} \leq \mathbf{Adv}_{\mathcal{R}_{\text{ANO-1PCA1}}}^{\text{ANO-1PCA}}.$$

References

- [JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.
- [MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.